Sentiment Analysis with Neural Models

Roee Tekoah Yona Gan Semester B, 2026

1 Setup

The Stanford Sentiment Treebank contains movie-review sentences and labelled constituent phrases. Sentiment values at most 0.4 are negative, values at least 0.6 are positive, and the middle band is discarded as neutral. The supplied split then has 7,693 full training sentences, 962 validation sentences, and 962 test sentences. Models 1 through 3 train on the full sentences and their labelled subphrases, giving 73,570 training examples. Validation and test contain only full sentences.

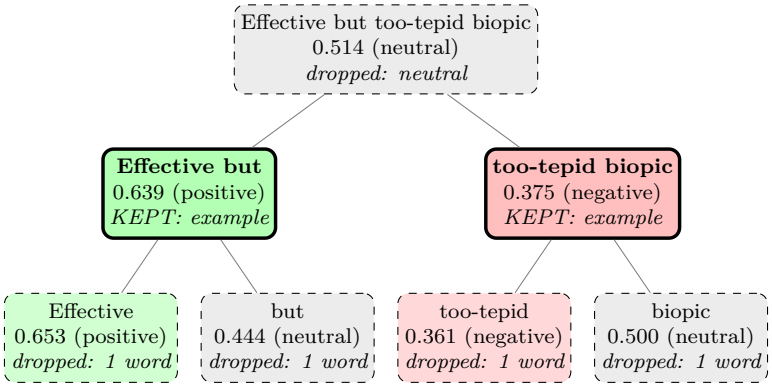


Figure 1: Phrase-level labels in one Stanford Sentiment Treebank sentence. The supplied loader keeps multi-word non-neutral constituents as training examples and drops neutral nodes and single-word leaves.

The negated-polarity subset contains the 62 test sentences with a root child whose polarity opposes the full sentence. The rare-word subset is the 50 test sentences with the lowest maximum training frequency among their sentiment-bearing words. The test set has 494 positive and 468 negative examples, so its majority-class baseline is 51.4%. The fine-tuned model instead uses 2,500 sampled full training sentences without subphrases.

Table 1: Prescribed training setup. The frozen embedding models use `distilroberta-base` token embeddings.

Model	Training input	Learning rate	Weight decay	Batch	Epochs	Other
One-hot log-linear	73,570 phrases	0.01	0	64	10	-
Embedding log-linear	73,570 phrases	0.01	0	64	10	frozen input
BiLSTM	73,570 phrases	0.001	0.0001	64	4	dropout 0.5
Fine-tuned Transformer	2,500 full sentences	1e-05	0	64	2	end-to-end

All experiments use seed 42. Every model is optimized with Adam and `BCEWithLogitsLoss`; prediction applies a sigmoid and uses a probability threshold of 0.5.

2 One-hot log-linear model

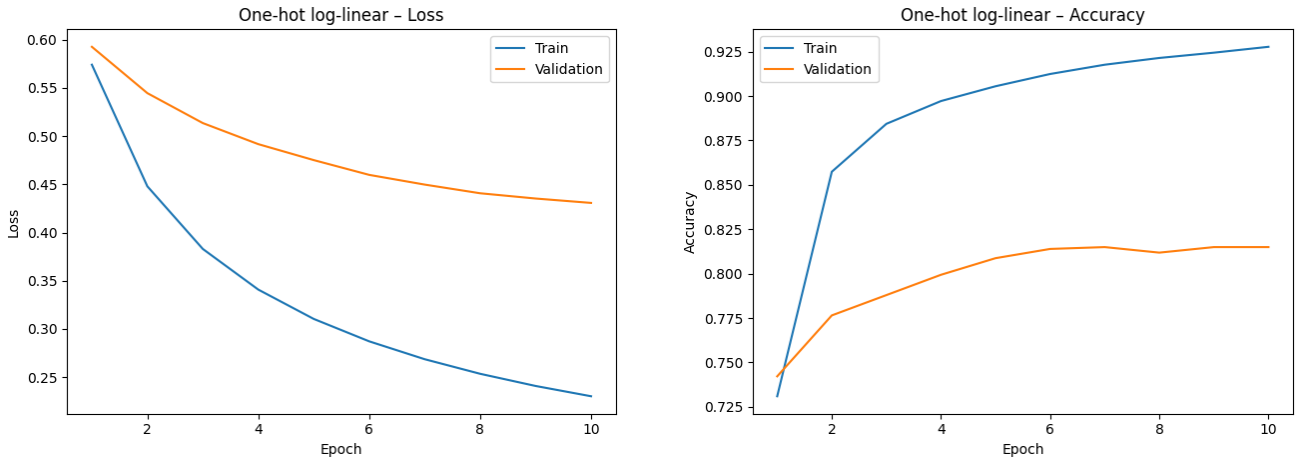
2.1 Model

Averaging one-hot vectors produces normalized word counts. If θ_w is the coefficient assigned to word w, the sentence logit is the average of its words' independently learned sentiment contributions plus the bias:

$$\bar{x} = \frac{1}{m}\sum_{i=1}^{m} \text{onehot}(w_i), \qquad z = \theta^\top \bar{x} + b = \frac{1}{m}\sum_{i=1}^{m} \theta_{w_i} + b.$$

The 16,271-word vocabulary contributes one adjustable coefficient per word; together with the bias, this gives 16,272 trainable parameters. Because this vocabulary is built from the whole corpus, it includes 1,667 words that never occur in training. Their one-hot coordinates are therefore always zero during training, so the corresponding classifier weights get zero gradient and remain at their initial random values.

This flexibility can build a detailed in-domain sentiment lexicon. It cannot share evidence between related words, however, and two sentences with the same word counts get exactly the same representation regardless of order.



(a) Loss. (b) Accuracy.

Figure 2: One-hot log-linear seed-42 learning curves. Train is the optimization-pass average; validation is evaluated after each epoch.

2.2 Results

Final validation accuracy is 81.50%; test accuracy is 82.85% with loss 0.4258. Negated-polarity accuracy is 66.13%, and rare-word accuracy is 58.00%.

2.3 Discussion

By the final epoch, optimization-pass training accuracy is about 93% while post-epoch validation accuracy remains around 81%; training-pass loss is about 0.23 versus 0.43. The model therefore fits training batches much better, consistent with memorization or overfitting. The sets differ, however: training includes many short labelled subphrases, while validation contains longer complete sentences whose labels depend on phrase interactions. The gap likely reflects both stronger training fit and harder validation examples, so it is not a pure generalization-gap measurement. Because validation loss keeps falling, later epochs are not shown to harm validation performance.

At 82.85% test accuracy, the model is a strong lexical baseline rather than a near-solution: its independent word coefficients capture many familiar sentiment cues, but the remaining 17.15% error rate shows that lexical evidence alone is insufficient for a substantial part of the task. Averaging also discards word order and phrase composition, so sentences with identical word counts get the same score even when their structure changes the sentiment.

3 Transformer-embedding log-linear model

3.1 Model

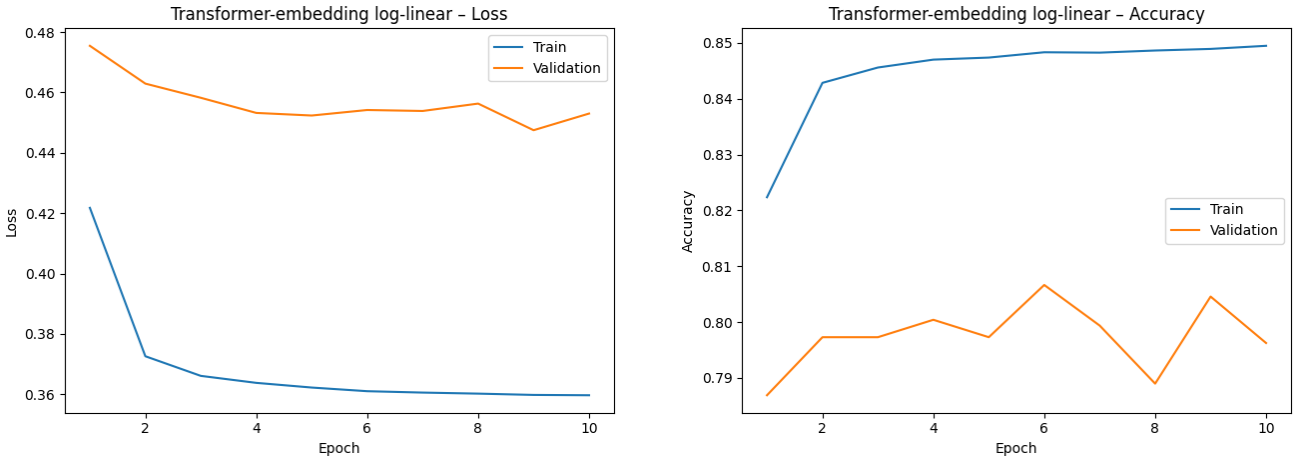
The tokenizer maps a sentence to subword IDs, and each ID retrieves one row from the pretrained `distilroberta-base` input embedding matrix. Special tokens are excluded from the average. The attention and feed-forward layers are never run, so the same token gets the same lookup vector in every sentence. "Pretrained" therefore does not mean "contextual" here; contextualization would require running the Transformer layers.

The learned vector θ maps every token vector to one scalar sentiment score. Linearity gives

$$z = \theta^\top \left(\frac{1}{m} \sum_i e_i \right) + b = \frac{1}{m} \sum_i \theta^\top e_i + b.$$

Thus the sentence logit is the average of its token scores plus the bias. The classifier need not assign meaning to individual embedding coordinates; it learns the one projection useful for sentiment. Only this projection and its bias are trained: 768 weights plus one bias give 769 trainable parameters.

An out-of-range token ID contributes a zero vector while still counting toward the average, as required by the specification. Because the tokenizer and embedding matrix come from the same `distilroberta-base` checkpoint, every generated token ID should correspond to a matrix row, so this fallback is not expected to affect the experiment.



(a) Loss.

(b) Accuracy.

Figure 3: Transformer-embedding log-linear seed-42 learning curves. Train is the optimization-pass average; validation is evaluated after each epoch.

3.2 Results

Final validation accuracy is 79.63%; test accuracy is 82.43% with loss 0.3941. Negated-polarity accuracy is 62.90%, and rare-word accuracy is 72.00%.

3.3 Discussion

Optimization-pass training accuracy reaches about 85% early and changes little, while training loss nearly stabilizes. Because the classifier is logistic regression over fixed features, its objective is convex in the learned weights and bias; the plateau is therefore more plausibly a limit of the fixed averaged representation than entrapment in a bad local minimum. The plotted training values are online averages, so this is qualitative evidence of saturation rather than an exact post-epoch ceiling.

The representation is fixed, non-contextual, and reduced to one average before classification. Pretrained sharing nevertheless lets a rare token obtain a useful score when its vector resembles familiar sentiment tokens, while BPE provides reusable units for unfamiliar word forms. The rare-word result supports this transfer mechanism. It does not constitute a separate measurement of accuracy on completely unseen tokens.

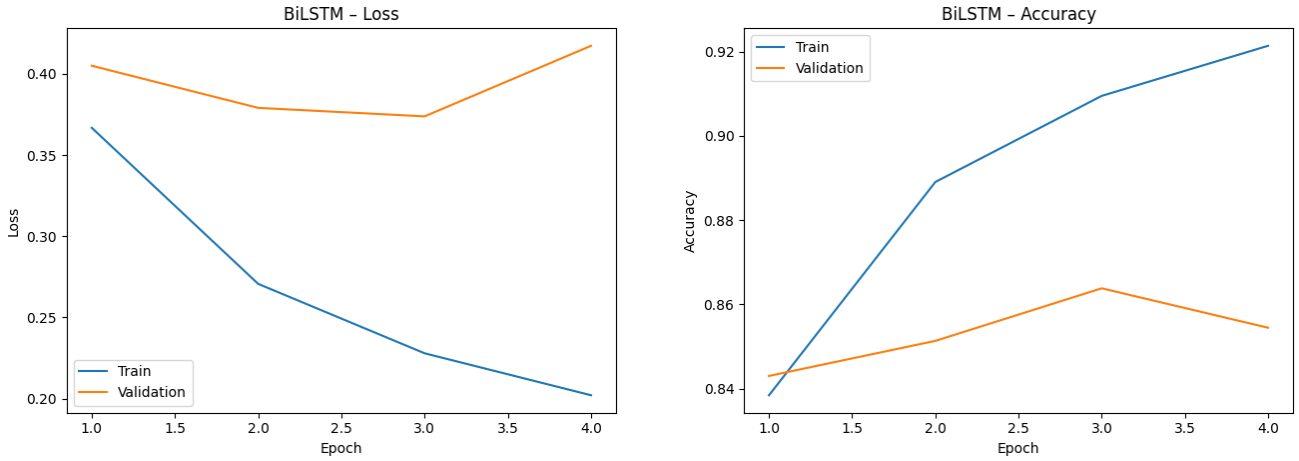
4 BiLSTM

4.1 Model

The input is an ordered sequence $e_1, \ldots, e_T$ of frozen token embeddings. Special tokens are excluded. The converter keeps the first 52 subword tokens and discards any later tokens; shorter sequences are padded with trailing zero-vector rows until their shape is 52×768. An out-of-range token ID also leaves a zero row. Thus truncation can remove sentence-final evidence, while padding provides a fixed batchable shape.

At each position, the forward LSTM updates its hidden and cell states from the current token vector and the earlier states, so its state summarizes an ordered prefix. The backward LSTM performs the analogous computation from right to left. Because sequences are not packed and no padding mask is passed to the LSTM, both directions process all 52 positions, including zero padding.

The final forward and backward hidden states, each 100-dimensional, are concatenated. Dropout 0.5 is applied before the final linear classifier. The BiLSTM and linear head contain 696,201 trainable parameters. The frozen lookup table and dropout add no trainable parameters.



(a) Loss.

(b) Accuracy.

Figure 4: BiLSTM seed-42 learning curves. Train is the optimization-pass average; validation is evaluated after each epoch.

4.2 Results

Final validation accuracy is 85.45%; test accuracy is 84.93% with loss 0.3877. Negated-polarity accuracy is 67.74%, and rare-word accuracy is 68.00%.

4.3 Discussion

Training-pass loss falls from epoch 3 to epoch 4 while validation loss rises, which is consistent with overfitting under the prescribed setup. The model is expressive enough to fit the training batches increasingly well, so its lower held-out accuracy should not be read as an inability to represent ordered patterns; it is a generalization result under this data and optimization budget.

Only the two final recurrent states reach the classifier. Because sequences are not packed, the recurrent layers also process zero-padding, which can affect final states according to sentence length and direction.

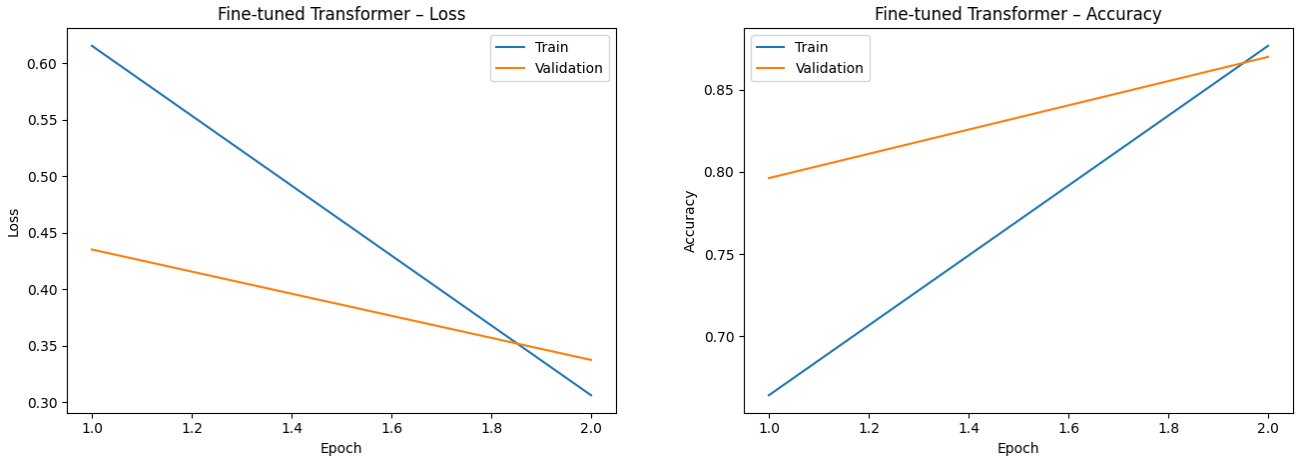
5 Fine-tuned Transformer

5.1 Model

The matching tokenizer keeps the special first token, truncates or pads each input to 64 tokens, and supplies an attention mask so padded positions do not participate in attention.

The full `distilroberta-base` encoder is run. At every layer, self-attention updates each token representation using information from other tokens, followed by the layer's remaining transformations. The classifier takes the final hidden state of the first special token, which has gathered sentence-level information through the Transformer layers, and maps it through a linear head to one sentiment logit. All backbone and head parameters are updated by the sentiment loss; the small learning rate 1e-05 limits the size of those updates. The backbone contributes 82,118,400 parameters and the 768-to-1 head adds 769, giving 82,119,169 trainable parameters in total.

The frozen-embedding log-linear model never runs these contextual layers. The BiLSTM builds context sequentially from frozen vectors. Fine-tuning instead starts with pretrained contextual layers and adapts the entire stack end-to-end.



(a) Loss.

(b) Accuracy.

Figure 5: Fine-tuned Transformer seed-42 learning curves. Train is the optimization-pass average; validation is evaluated after each epoch. Training uses 2,500 full sentences for two epochs.

5.2 Results

Final validation accuracy is 87.01%; test accuracy is 88.05% with loss 0.2902. Negated-polarity accuracy is 74.19%, and rare-word accuracy is 78.00%.

5.3 Discussion

Even under the prescribed supervised budget of only 2,500 full sentences and 2 epochs, the fine-tuned Transformer has the strongest validation, test, negation, and rare-word results in this run. It is not learning language from those labels: pretraining already supplies broad lexical, syntactic, and contextual regularities. Sentiment training adapts that representation and teaches the classification head; each update can change the contextual layers rather than only a newly initialized sentiment lexicon.

Validation is still improving at epoch 2, so the experiment demonstrates strong performance within the prescribed budget but does not establish that two epochs are optimal or sufficient.

6 Model comparison

Table 2: Final results from the canonical seed-42 run. Accuracies are percentages; the special subsets contain 62 and 50 examples.

Model	Final val. acc.	Test acc.	Test loss	Negated	Rare
One-hot log-linear	81.50	82.85	0.4258	66.13	58.00
Embedding log-linear	79.63	82.43	0.3941	62.90	72.00
BiLSTM	85.45	84.93	0.3877	67.74	68.00
Fine-tuned Transformer	87.01	88.05	0.2902	74.19	78.00

6.1 The two log-linear models

The one-hot model has higher final validation accuracy (81.50% versus 79.63%) and slightly higher test accuracy (82.85% versus 82.43%). Averaged one-hot inputs are normalized word counts, so the classifier learns an independent sentiment coefficient for every vocabulary word. This simple lexical baseline is already strong when familiar words reveal the label.

The embedding model trades that in-domain flexibility for pretrained sharing: one projection scores all token vectors, allowing semantically or morphologically related tokens to transfer evidence. Overall held-out accuracy is close, but rare-word accuracy rises from 58.00% to 72.00%. BPE also represents unfamiliar forms, although no separate unseen-token accuracy was measured. Both averages still discard order and composition.

6.2 BiLSTM and fine-tuned Transformer

The BiLSTM improves test accuracy by 2.08 points over one-hot and 2.50 points over embedding log-linear. Order therefore helps, but its effect is modest: the BiLSTM still builds context sequentially from frozen embeddings and compresses the sentence into two final states. Its small negation-subset gain also shows that restoring order alone does not solve composition.

The fine-tuned Transformer reaches 88.05% test accuracy. Its pretrained self-attention layers let every token incorporate information from other tokens, and all six contextual layers are adapted end-to-end. The comparison therefore reflects both architecture and pretraining, not a pure difference in model expressivity.

6.3 Special subsets

On negated polarity, the fine-tuned Transformer is highest at 74.19% and the embedding log-linear model is lowest at 62.90%. Averaging cannot encode which word a negator modifies; the BiLSTM can, but the effect must traverse gated states and survive endpoint compression. In attention, the weight between two tokens depends on their query-key match, so a target can mix the negator's value directly before the sentence state aggregates the contextualized phrase. Pretraining supplies these operations and fine-tuning specializes them. The 6.45-point gain over BiLSTM is consistent with this mechanism, not proof of it. Accuracy is still only 74.19%, and 62 examples are too few for a general ranking.

On rare words, the fine-tuned Transformer is highest at 78.00% and one-hot is lowest at 58.00%. Pretrained vector sharing and BPE help all embedding models. Because the BiLSTM uses the same frozen table, the Transformer's advantage reflects contextual disambiguation and end-to-end adaptation, not coverage alone. The subset has only 50 examples.

6.4 Computational cost

Table 3: Model-forward cost on an NVIDIA GeForce RTX 4060 Ti. Times are amortized batch-64 throughput from the audited benchmark.

Model	Trainable parameters	Additional required model asset	μs/sentence
One-hot log-linear	16,272	Vocabulary/index map; one-hot construction excluded	0.5
Embedding log-linear	769	Frozen 50,265 × 768 table (≈ 154 MB FP32)	0.4
BiLSTM	696,201	Same frozen table (≈ 154 MB FP32)	10.8
Fine-tuned Transformer	82,119,169	Full backbone and head (≈ 328 MB FP32 values)	658.2

The fine-tuned Transformer buys its accuracy advantage with much greater forward computation and storage; the BiLSTM lies between it and the log-linear models. Counting only FP32 parameter values, inference storage is approximately 0.065 MB for the one-hot linear layer, 154.4 MB for embedding log-linear, 157.2 MB for BiLSTM, and 328.5 MB for the Transformer. The first number excludes vocabulary metadata; the middle two include the frozen lookup table.

The embedding model offers strong rare-word improvement at low forward cost; the fine-tuned model gives the best results at much greater cost. Preference depends on accuracy, storage, throughput, and simplicity. Timings are hardware-specific batch-64 model-forward throughput, not batch-1 or end-to-end latency; preprocessing, transfer, and loading are excluded.

7 Parameter counting

Following the question, let d be model width, V vocabulary size, n context length, and L layer count. Biases, the classification head, positional embeddings, feed-forward layers, and LayerNorm are ignored. Attention is treated as one head with four $d \times d$ projections.

7.1 Transformer

The token embedding table contains one learned d-dimensional vector for each of the V token types, so

$$\boxed{P_{\text{embedding}} = Vd}.$$

Every attention layer contains four learned $d \times d$ projections, W_Q, W_K, W_V, W_O, so across L layers

$$\boxed{P_{\text{attention}} = 4Ld^2}.$$

The whole encoder count under the stated assumptions is therefore

$$\boxed{P_{\text{Transformer}} = P_{\text{embedding}} + P_{\text{attention}} = Vd + 4Ld^2}.$$

For $V = 30{,}000$ and $d = 512$,

$$\boxed{P_{\text{embedding}} = 30{,}000 \times 512 = 15{,}360{,}000}.$$

One attention layer contributes

$$\boxed{4 \times 512^2 = 1{,}048{,}576 \text{ parameters}}.$$

The embedding table and all attention projections are equal in size when

$$Vd = 4Ld^2, \qquad L = \frac{V}{4d} \approx 14.6.$$

Therefore, for 14 or fewer layers the embedding table is larger, while from 15 layers onward the combined attention projections are larger. Neither parameter count depends on context length n, because the same projection matrices are reused at every token position. A longer context increases computation and temporary memory because the attention-score matrix has size $n \times n$, but that matrix is computed for each input rather than learned.

7.2 RNN

If each of L recurrent layers has R learned parameters, the final count is

$$\boxed{P_{\text{RNN}} = Vd + LR}.$$

This is also independent of n: one recurrent layer applies the same transformation at every time step. A longer sequence increases the number of sequential computations, not the number of learned values.

7.3 Character tokenization

With characters, V falls from about 30,000 to 100, so Vd shrinks 300 times (from 15,360,000 to 51,200 at $d = 512$). A roughly fivefold longer context changes computation, not parameter counts:

$$\boxed{P_{\text{Transformer}} = 100d + 4Ld^2, \qquad P_{\text{RNN}} = 100d + LR}.$$

Neither $4Ld^2$ nor LR changes, because neither contains V or n. Thus both models have fewer parameters: only the embedding-table term shrinks, while the longer context increases computation but not the number of learned parameters. For the Transformer, the attention stack now dominates.

The cost appears elsewhere. Five times as many positions makes the Transformer attention-score computation and memory about 25 times larger, while the RNN performs about five times as many sequential steps. Character models also must learn how characters form useful lexical units, work that subword tokenization partly supplies in advance.